PROJECT: Cloud Education Database and WordPress Server Implementation

Role: Cloud database designer, Linux server administrator, and WordPress implementer

Tools: AWS EC2, Ubuntu Server, Apache, MySQL, phpMyAdmin, WordPress, Terminus, SSH, PHP shortcodes, ERD modeling

Focus Areas: Relational database design, cloud hosted web application deployment, access control, student data workflow, troubleshooting, and implementation documentation

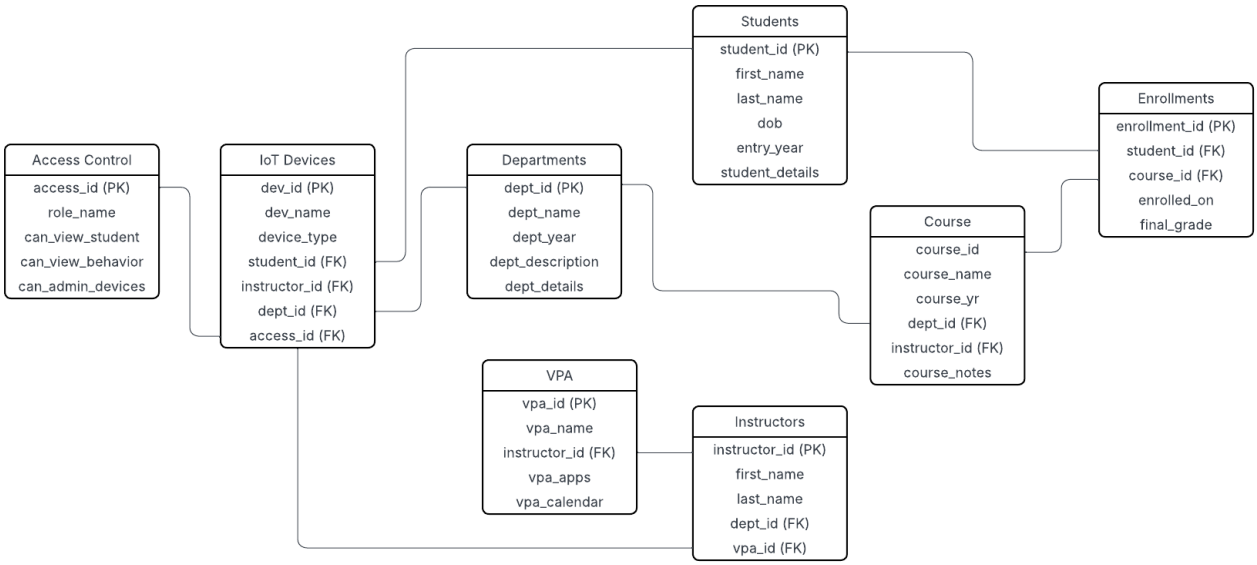

Figure 1. YODAE relational data model showing student, course, instructor, department, IoT device, VPA, enrollment, and access control relationships.

Overview:
This project implemented a cloud hosted educational support system for the YODAE scenario. The solution combined an entity relationship diagram, SQL tables, a WordPress front end, and a basic student data workflow hosted on an AWS EC2 Ubuntu server. The goal was to connect database design decisions to a working web application that could support K to 12 educators, administrators, students, IoT classroom devices, and virtual personal assistants.

Client Mission Alignment
YODAE was modeled as a cloud education provider that needed scalable data management and reliable classroom technology support. The database structure was designed to record academic relationships, device ownership, department oversight, and access privileges while leaving room for future growth in student, course, and device data. This aligns with cloud education concepts that emphasize flexible infrastructure, online learning access, and secure educational service delivery.

Systems Challenge & Constraints:
The main challenge was translating a conceptual education environment into a relational schema and then validating that design through a functional WordPress deployment. The implementation had to account for student enrollment, instructor assignment, course ownership, department structure, IoT device mapping, VPA associations, and access control. The build also required server recovery work because the WordPress configuration was not completed in a single session, creating a need to reenter the server through SSH and repair the environment before continuing.

Technical Constraints
The project was constrained by manual command entry, WordPress publishing behavior, shortcode formatting, table naming consistency, and the need to keep the web pages connected to the SQL database. Small syntax or spelling issues created visible application errors, which reinforced the importance of naming conventions, repeatable configuration steps, and controlled implementation notes.

Architecture and Data Model

The relational architecture decomposed the YODAE environment into Students, Instructors, Courses, Departments, Enrollments, IoT Devices, Virtual Personal Assistants, and Access Control. Each table used primary keys and foreign keys to keep the educational workflow traceable from the student record to course enrollment, instructor ownership, department alignment, and device authorization. This structure supported both academic operations and secure classroom technology management.

Implementation Approach

The implementation followed a practical cloud application sequence. First, the WordPress server was restored and verified through command line access. Next, file permissions and configuration files were adjusted so Apache and WordPress could operate correctly. After the server was reachable, the database tables were created in MySQL through phpMyAdmin, then WordPress pages and PHP shortcodes were used to connect front end pages to database actions.

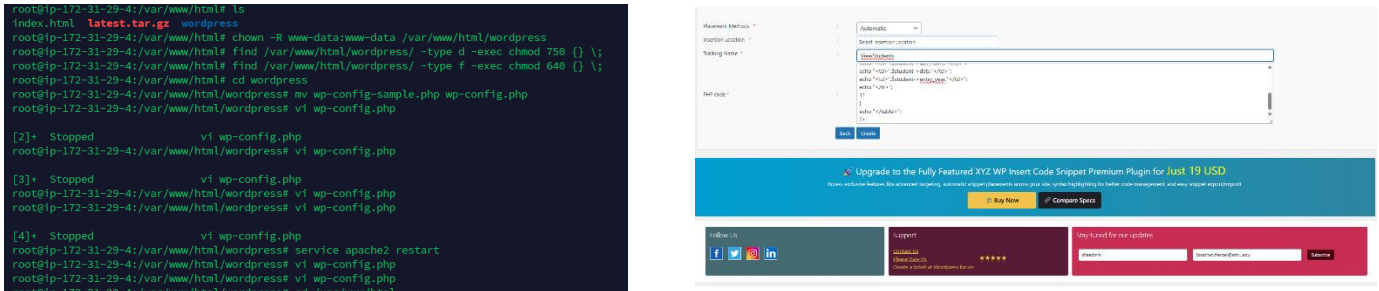

Figure 2. Server recovery and WordPress configuration work performed through SSH, followed by shortcode configuration for the student view workflow.

Server Configuration and Recovery

The server recovery process used Terminus and SSH to reconnect to the Ubuntu instance, verify the WordPress directory, adjust file ownership, apply directory and file permissions, edit wp-config.php, restart Apache, and redownload the WordPress package when needed. This work demonstrated operational troubleshooting rather than a simple static page build. The server had to be brought back into a stable state before the application layer could be tested.

WordPress Shortcode and Data Entry Workflow

The WordPress implementation used a PHP shortcode to pull student data into the View Students page. This reduced the amount of manual page coding required and created a reusable link between the WordPress interface and the database query logic. During testing, the publishing feature produced invalid JSON response errors, which required repeated configuration checks and reinforced the need for clean syntax, stable plugins, and careful copy and paste validation.

SQL Database Validation

The SQL implementation validated the ERD by creating the required tables inside the WordPress database environment. The phpMyAdmin view confirmed that AccessControl, Course, Departments, Enrollments, Instructors, IoTDevices, Students, and VPA tables were present. This confirmed that the logical design was translated into physical database objects. During validation, spelling inconsistencies became a defect source, showing that database naming standards matter because front end queries depend on exact table and field names.

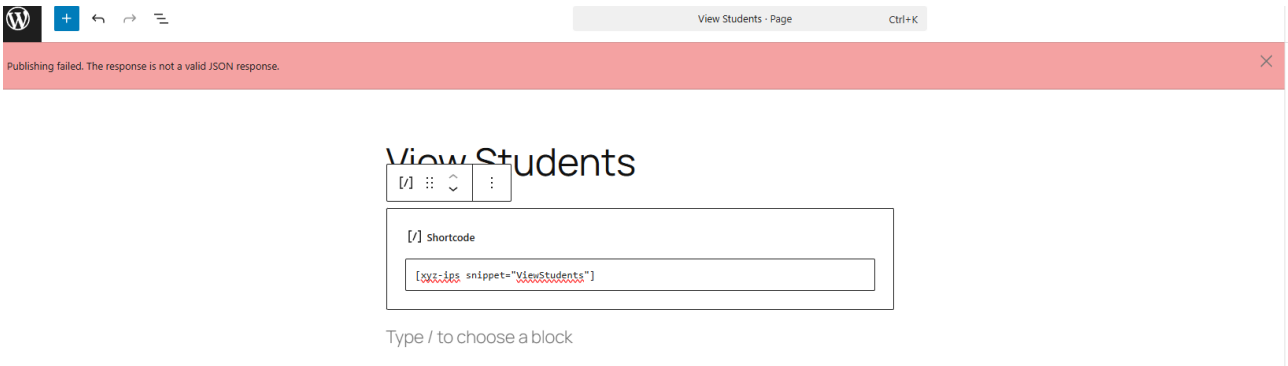

Figure 3. phpMyAdmin validation of the SQL tables created from the ERD.

Application Demonstration

The WordPress front end included an Add Student page and a View Students page. The Add Student page accepted first name, last name, date of birth, and entry year values. The View Students page then displayed the newly created record, proving that the front end workflow could write and read student data from the backing database.

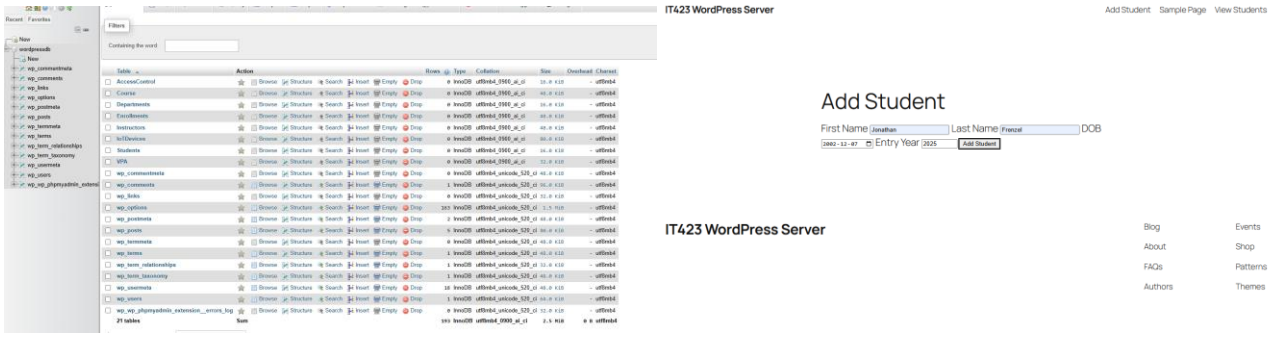

Figure 4. WordPress Add Student form and View Students confirmation page showing database backed application behavior.

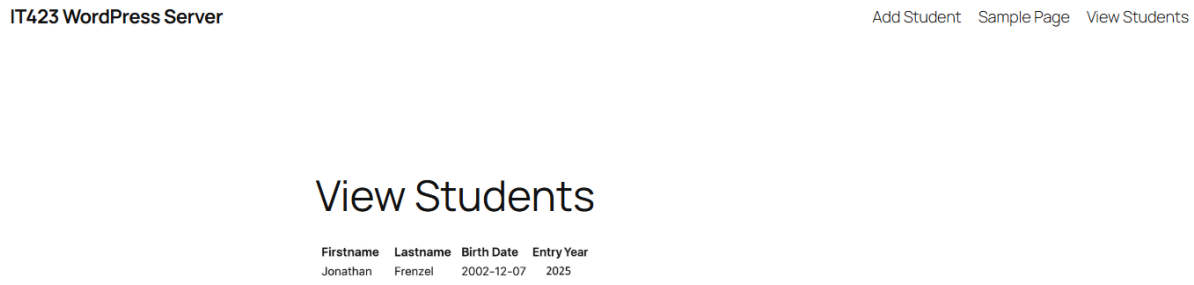

Figure 5. Public WordPress site route demonstrating the deployed application path on the AWS EC2 host.

Design Decisions and Tradeoffs

The design prioritized a normalized relational structure over a flat data model so student records, course data, departments, devices, and access rights could evolve independently. This increased implementation complexity but improved maintainability and made the system easier to scale. WordPress shortcodes were used instead of building a full custom theme because they provided a faster path to functional data entry and display.

Outcomes and Impact

The final implementation produced an ERD, created SQL tables, restored and configured a cloud hosted WordPress server, built student data pages, and demonstrated a working record creation and view workflow. The project showed the full path from system design to cloud application deployment, including infrastructure troubleshooting and database validation.

Lessons Learned

This project reinforced the importance of exact naming, repeatable setup procedures, and early validation. A small spelling issue or malformed shortcode can break the application even when the database design is sound. The work also showed that cloud systems administration requires both design thinking and hands on recovery skills when services, permissions, or application components fail.